

An Empirical Study on Configuration Robustness of Unsupervised IDS

Omar Anser* Jérôme François*[†] Isabelle Chrisment[†]

*SnT — University of Luxembourg, Luxembourg

{firstname.lastname}@uni.lu

[†]Inria, Université de Lorraine, CNRS, LORIA, Nancy, France

{firstname.lastname}@inria.fr

Abstract—Machine learning based anomaly detection intrusion detection systems (AD IDSs) often rely on a hyper-parameter setting that is chosen once and never revisited. Moreover, when network traffic drifts, the detector’s ability to separate normal from malicious activity can decline significantly. Although re-training is usually recommended, it remains unclear how large the performance degradation is when retraining is performed without re-tuning the detector’s hyper-parameters. This paper assesses the robustness loss due to a non-optimized hyper-parameter configuration. We present a benchmarking method that isolates different types of context drift (topology, traffic patterns) in a dataset alongside a robustness metric evaluation. We apply our method to the daily-partitioned CIC-IDS-2017 and CSE-CIC-IDS-2018 datasets, covering two realistic scenarios: day-to-day drift and cross-year reuse of a configuration for the same attack family. Against an oracle tuned by Bayesian optimization, we evaluate several low-budget alternatives and simple configuration-transfer schemes. None produces satisfactory results, with the ROC-AUC dropping by roughly one third.

Index Terms—ML, AD IDS, Hyper-parameter, Context Drift

I. INTRODUCTION

Machine-learning-based anomaly-detection intrusion-detection systems (AD IDSs) learn a model of normal network behavior and trigger an alert whenever observed traffic deviates from this baseline [16]. Because they do not rely on labeled attack data, these systems remain effective even when adversaries use machine-learning techniques to mutate known patterns. To bypass such a system, an attacker must mimic benign behavior, which is a significantly more difficult task [17], [18]. Therefore, assuming there is sufficient benign data for training, AD IDSs are more resilient than signature- or model-based supervised detectors to previously unseen attacks and their variants.

Network traffic, however, is anything but static. The mix of protocols, destinations, and user actions shifts from day to day, and the statistical profile shifts again when new software, hardware or policies are introduced [19], [20]. AD IDSs must thus adapt to variability in legitimate traffic while maintaining their intrinsic robustness to unseen attacks.

As a solution, AD IDSs can be retrained regularly, under the assumption that collecting or generating legitimate traffic is easier than attack traffic. Yet every anomaly detector still relies on multiple hyper-parameters (HPs): as for example, the number of trees in an Isolation Forest [21], its subsampling rate, its expected contamination fraction, etc. These HPs are

not learned by the algorithm itself. They must be fixed as a preamble manually or by an iterative tuning process before the AD IDS is fit with legitimate traffic. In practice, network administrators choose those settings once or copy the defaults suggested by a software library and then leave the configuration (i.e. the hyper-parameter values) unchanged [10], [21]. Limited time and expertise in machine learning make regular re-tuning impractical in operational networks.

Earlier work showed that reusing a fixed HP configuration harms detection capabilities [22], [23], but no study has measured how performance erodes, or how large the loss can be. Without such evidence, operators lack a principled basis for deciding how often to reconfigure or how much automation is worth the cost. This challenge is complementary to retraining: while retraining can often proceed without reconfiguring hyperparameters, reconfiguration inherently requires retraining to remain effective.

This paper addresses the open question *how large the performance loss can be due to optimized HP setting* through an empirical study based on the day-partitioned IDS2017 and IDS2018 datasets [24].

For each day in the dataset, we build an “oracle” baseline by leveraging Bayesian optimization (a common AutoML technique) to yield optimal configuration. However, this approach introduces a significant computational overhead and ideally requires access to labeled attack traces to guide the search. In this paper, by focusing on assessing the impact of configuration drift, we indirectly evaluate how frequently such optimization would be necessary in practice and whether simple alternatives could be leveraged.

We thus evaluate three cost-saving alternatives for an AD IDS: (i) the library defaults shipped with the software, (ii) a single global configuration tuned once on the union of all days, and (iii) a fast re-tuning that searches only a ten-percent subsample of the day’s flows. The daily experiment also assesses how performant an oracle (HP setting) tuned for a given day remains accurate for another day. This allows us to reveal the effect of short-term drift.

A second assessment investigates cross-year same-attack transferability by applying a configuration that is optimal for a given attack family in 2017 unchanged to the matching attack family in 2018.

In a nutshell, the contributions of this paper are as follows:

TABLE I: Overview of related work on configuration robustness and IDS configuration.

Research Area	Primary Focus	Representative citations	Relation to our study
HP Importance (ML literature)	Demonstrating that only a small subset of HPs significantly impacts model performance.	[1], [2]	Motivates quantifying per-HP impact before choosing what to tune; based on related work, our experiments later prioritize a set of critical HPs for the AD IDS under study.
Robustness to Data Distribution Shift (ML literature)	Developing methods to find HPs configurations that are robust to temporal data distribution shifts.	[3]–[9]	These are general-purpose solutions. Our paper first quantifies the necessity of such solutions specifically for AD IDS.
HPs Tuning for IDS	Emphasizing rigorous HPs selection/validation to ensure reliable IDS performance	[10]–[13]	Confirms the problem is critical in the IDS field. Our work provides the empirical cost of not re-tuning.
Neural Architecture Search (IDS-specific)	Automatically designing the architecture of deep learning detectors, not just tuning their HPs.	[14], [15]	Related but distinct; we keep the architecture fixed and study HPs tuning.
This Study (Identified Gap)	Quantifies degradation from fixed, non-optimized HPs on AD IDS as traffic evolves.	This paper	Provides empirical evidence for the re-configuration need highlighted above.

- A daily and cross-year benchmark that isolates the impact of fixed HP configurations under context drift in anomaly-based IDSs.
- An experimental comparison of three low-budget configuration strategies against a Bayesian-optimised oracle, including an analysis of day-to-day and same-attack transfer.
- Quantitative evaluation demonstrating that outdated or borrowed configurations reduce ROC-AUC by about one-third, motivating continuous automated re-configuration.

While our work is evaluated assuming Isolation–Forest, the methodology is still applicable to any other anomaly-based algorithm.

The rest of the paper is organized as follows. Section II formalises the configuration problem with notation. Section III discusses practical implications and related work. Section IV describes the two datasets and the evaluation methodology. Our experimental results are described in Section V. Section VI concludes the paper.

II. PROBLEM DEFINITION

Given a network environment with its hosts and activities, we assume that traffic traces are collected at one or more vantage points (monitoring locations) and subsequently aggregated into fixed intervals such as hours or days.

Our problem formulation follows a standard anomaly detection (AD) pipeline. As a core of any ML technique, there is the training phase. Each observation interval yields a single, unlabeled benign trace which serves as the training set:

$$X = \{x_1, \dots, x_n\} \subset \mathbb{R}^m, \quad (1)$$

where every vector x_i represents one network flow encoded by m numerical features (e.g., flow duration, total packet count, total bytes, mean packet size). Thus, a trace contains many flows. This unsupervised approach, training only on benign data, is fundamental to anomaly detection (AD) for IDS, as it enables the detector to identify novel attacks not present in any labeled training set.

An anomaly detector is defined as a real-valued scoring function:

$$h_\theta : \mathbb{R}^m \longrightarrow \mathbb{R} \quad (2)$$

configured by $\theta \in \Theta$ that comprises the set of HPs of the model. For an Isolation Forest, for example, θ includes the number of trees, the subsample size, and other algorithm-specific HPs.

Once trained, in the evaluation phase, the configured detector is then evaluated on a labeled test set $(\tilde{X}, \tilde{y})$ collected at a later interval in the same network, where $\tilde{y}$ assigns a binary label for every flow (1=attack, 0=benign).

To ensure high performance, rather than using default HP values, a configuration phase is introduced to adjust θ . This process, defined as the configuration procedure $\mathcal{C} : X \rightarrow \Theta$, typically involves an iterative loop of training and testing candidate configurations.

The data available at configuration time dictates the strategy. If attack traces are present, $\mathcal{C}$ can use a separate validation set (along with the benign trace) to guide HP selection. When no attack data exist, $\mathcal{C}$ relies solely on the benign trace and is tuned to respect a target false-positive budget.

Although defined as a scoring function, the anomaly detector ultimately outputs a binary decision. The score given by the equation (2) is transformed into a binary decision according to a threshold τ .

However, τ is *not* included among the variables optimized when selecting θ , because robustness is quantified over all possible thresholds. Specifically, robustness is measured as the area under the receiver operating characteristic (ROC) curve:

$$p_{\text{robust}}(h_\theta, \tilde{X}, \tilde{y}) = \text{AUC}(h_\theta(\tilde{X}), \tilde{y}), \quad (3)$$

which quantifies how well the detector ranks attacks ahead of benign flows without committing to a specific alert threshold (i.e. by averaging over all possible thresholds).

The configuration task is therefore:

$$\theta^* = \arg \max_{\theta \in \Theta} p_{\text{robust}}(h_\theta, \tilde{X}, \tilde{y}), \quad (4)$$

and the corresponding optimal robustness, p_{robust}^* , serves as a well-tuned empirical baseline against which all practical

configuration strategies will be compared in the experimental study.

III. RELATED WORK

Table I synthesizes the related work and contextualizes our study's specific contribution.

Hutter, Hoos and Leyton-Brown introduce *fANOVA* [1], a tool that uses random-forest models to measure how much each HP influences performance. Across many machine-learning and optimization tasks, they found that only a small set of HPs accounts for most of the variation in results. Similarly, Van Rijn and Hutter [2] reached the same conclusion through a larger-scale study using 100 OpenML benchmarks.

While these studies establish the importance of HPs, a second major research area concerns the stability of a configuration as data evolves after the model has been trained. Indeed, in DOMAINBED [25], WILDS [26] and HYPERTIME suite [27], the authors show that a full reconfiguration of HPs is necessary to achieve the best performance.

However, to avoid the costly optimization of HPs, several works aim to make the configuration more robust. A possible solution is to perturb the HPs tuning process by injecting new data that increase the robustness of the resulting configuration [3], [4]. More recently, *distributionally robust Bayesian optimization* (DRBO) [5] and its variant, AIRBO [6] and LOCBO [7], search for hyperparameter configurations whose performance degrades least across plausible shifts. Moreover, AUTOFT selects HPs maximizing the accuracy on a small out-of-distribution validation split [8]. For streaming data, Veloso *et al.* continuously apply small modifications to HP configurations and keep the variants with the lowest error [9], constituting the basis for a new optimization search when a drift in data occurs.

All these work intends to assess the depth of the re-configuration problem for ML algorithms and provide solutions, but none of them particularly consider the context of IDSs.

In this perspective, recent work in security-based machine learning emphasizes that rigorous HP selection is critical for reliable intrusion-detection. Arp *et al.* [10] list *Biased Parameter Selection* as a common pitfall: tuning on test data overestimates reported accuracy and results in unstable deployments; hence they advocate a strict validation split or cross-validation before any test-set use [10]. Pendlebury *et al.* reach a complementary conclusion with the TESSERACT benchmark [11], demonstrating that the training-set malware ratio is a decisive HP; an automated search for it significantly improves long-term performance. Masum *et al.* [12] show that carefully tuning a deep-learning IDS with BO largely improves detection accuracy well beyond a random-search baseline. In a fully unsupervised setting, Anser *et al.* [13]'s framework jointly infers detector HPs and decision thresholds using benign traffic only. The method bootstraps from pre-optimized configurations and refines them through a meta-learning routine.

Beyond tuning hyperparameters, several authors have applied neural-architecture search (NAS). Lin *et al.* present MENAS [14], a multi-objective evolutionary NAS framework that explores convolutional neural-network (CNN) architectures for intrusion detection, aiming simultaneously to boost accuracy and to reduce the computations required per input. Shao *et al.* embed a genetic NAS algorithm within a federated-learning loop, producing FED-GA-CNN-IDS [15], a lightweight CNN architecture tailored to industrial-control environments.

Despite these advances, the impact of *fixed HPs* chosen once and left unchanged as traffic evolves on anomaly-based intrusion-detection systems remains underexplored. Our study quantifies this overlooked source of degradation.

IV. DATASETS AND EVALUATION METHODOLOGY

A. Datasets

Our empirical analysis builds on the widely used IDS2017 and IDS2018 benchmarks [24]. We use a corrected version [28] in which errors, and mislabeled traces have been fixed. Both datasets were collected at the Canadian Institute for Cybersecurity's testbed, which mirrors an enterprise network and allows the controlled injection of attacks, while benign background traffic, predominantly HTTP and HTTPS flows, was generated by agents that replay patterns learned from real user activity.

IDS2017 offers five daily traces collected; the first is purely benign and is therefore discarded, leaving four operational days that mix legitimate flows with fourteen attack families. IDS2018 was recorded eleven months later by the same research lab using a cloud-based testbed consisting of seven networks and about 500 virtual machines. Consequently, the network topology differs from IDS2017. This dataset includes ten daily captures that introduce further attack variations. Since data collection concludes at the end of each day and resumes only after an unobserved interval, the dataset consists of day-level traces rather than a continuous stream. Our evaluation therefore compares performance across daily traces.

TABLE II: Features considered in each dataset

Feature Name	Description
Src Port	Source service port numbers (one-hot encoded).
Dst Port	Destination service port numbers (one-hot encoded).
Protocol	Network protocol of the flow (one-hot encoded).
Flow Duration	Duration of the flow.
Total Fwd/Bwd Packets	Packets in the forward/backward direction.
Total Length of Fwd/Bwd Packet	Size of packets in forward/backward direction.
Flow Bytes/s	Bytes transferred per second.
Flow Packets/s	Packets transferred per second.
Fwd/Bwd IAT Total	Time between packets in fwd/bwd direction.
Fwd/Bwd Packets/s	Forward/backward packets per second.

For clarity, we aggregate the fourteen daily captures as $D_0, \dots, D_{13}$; the four IDS2017 days are D_i , $i \in [0, 3]$, while the ten IDS2018 days are D_i , $i \in [4, 13]$. Each record represents a flow identified by the standard five-tuple (source and destination IP addresses and ports and the protocol). These unidirectional flow are then aggregated into bidirectional flows. Originally, 83 features were included such as

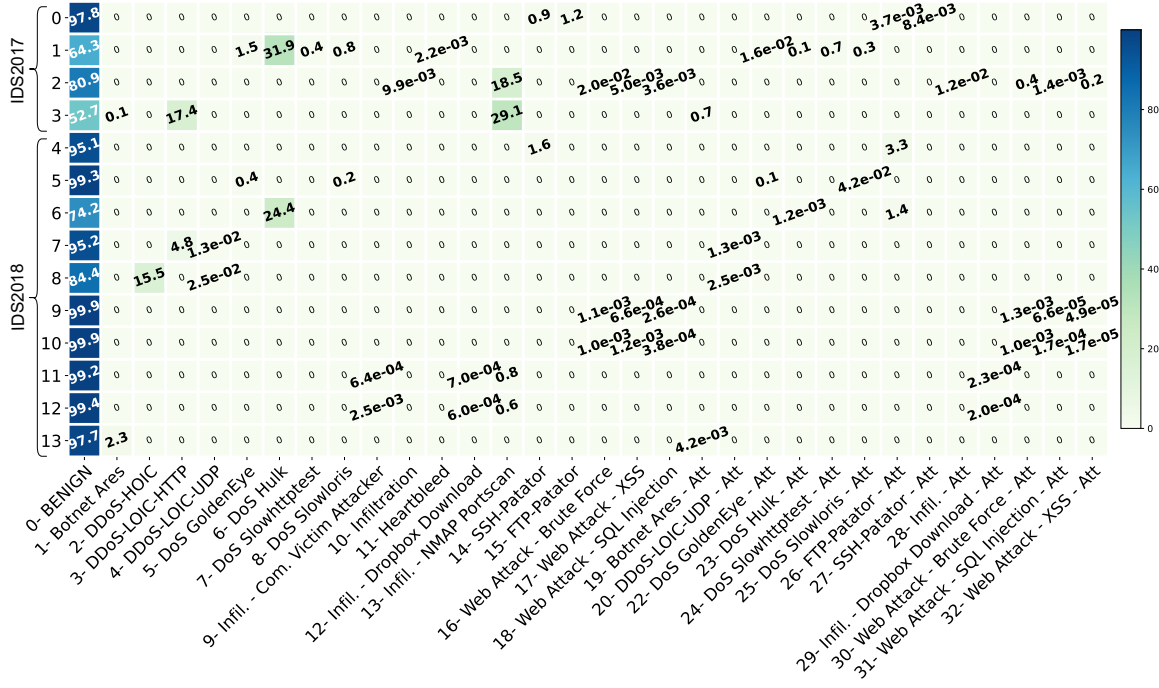


Fig. 1: Daily traffic composition in IDS2017 (D0–D3) and IDS2018 (D4–D13). Rows denote days and columns denote traffic classes (benign and 31 attack families), ordered by taxonomy ID. Cell color shows the class share per day (% of flows) using a log-scaled color mapping; “Att.” denotes unsuccessful attack attempts.

packet counts, byte counts, timing statistics, TCP flags, and so on. We start from the pre-cleaned version prepared by Liu *et al.* [28], which removes malformed and duplicate entries. We follow the original authors’ preprocessing pipeline [24] and keep the fourteen attributes listed in Table II. Prior work shows that these features capture most of the discriminative power while reducing redundancy and sparsity [29], [30]. All numeric features are subjected to min–max normalisation, whereas the categorical attributes (destination port, source port and protocol) are one-hot encoded using their 20 most frequent values; all less frequent values are merged into an “other” category.

Figure 1 visualizes the daily composition of benign traffic and 31 distinct attack families. Rows correspond to the fourteen capture days D_0 – D_{13} ; columns correspond to traffic classes ordered by their taxonomy ID. Color intensity encodes the percentage share of each class within the day, so darker cells mark higher prevalence. A label beginning with “Att” denotes an attempted (unsuccessful) attack (for example, failed SQL injections or half-open TCP handshakes).

As show, benign flows dominate every day, frequently exceeding 80 % of the total volume, which mirrors the imbalance faced by real-world anomaly detectors. Second, high-volume attacks such as DDoS and brute-force login (taxonomy IDs 4, 14, 15) still account for less than 2 % of daily flows, showing that these types of attacks remain statistically infrequent.

Giving that each dataset was generated in a different network environment (network topology and/or number of hosts),

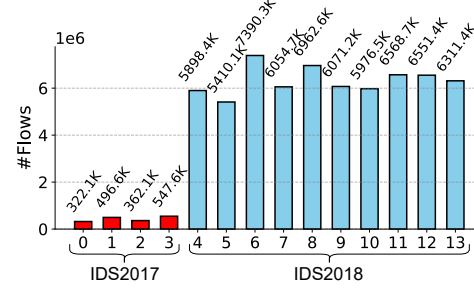


Fig. 2: Flow Count Across Days

Figure 1 also shows that for a given environment, an attack is generated only once, *i.e.* the mix of attacks and benign load varies enough that no two days exhibit identical distributions. Hence, each day constitutes a unique context for anomaly detection. The only near-repetitions occur in IDS2018, where Web attacks appear on both D_9 and D_{10} , and Infiltration traffic on D_{11} and D_{12} .

Figure 2 complements this view with a bar chart of bidirectional flow counts, confirming that traffic volume itself drifts substantially, from well below 200 000 flows on the least active 2017 day to more than 600 000 on several 2018 days, a threefold change in daily load.

Therefore, using these datasets provide a justified basis for evaluating how a configuration remains robust even if the nature of the traffic changes.

TABLE III: Isolation Forest HP ranges used by the oracle tuner. Type markers: ^{*}integer, [†]real-valued, [‡]log-uniform.

ID	HP	Search range	Practical effect
Θ_1	Sample size [*]	$[2^8, 2^{13}]$	Flows per tree; smaller is faster but may miss rare benign patterns.
Θ_2	Number of trees [*]	$[50, 500]$	More trees reduce score variance at higher cost.
Θ_3	Maximum depth [*]	$[8, 64]$	Shallow trees isolate earlier and score moderate deviations higher.
Θ_4	Split-feature fraction [†]	$[0.10, 1.00]$	Fraction of the m features tested at each split; affects overfitting.
Θ_5	Linear-combination probability [†]	$[0, 0.5]$	Enables random linear projections to capture multivariate structure.
Θ_6	Gain threshold [‡]	$[10^{-5}, 10^{-1}]$	Minimum gain to keep splitting; controls granularity in noisy regions.
Θ_7	Number of projections tested [*]	$[1, 100]$	Candidate projections evaluated before the best split is chosen.

B. AD IDS Design and Implementation

The anomaly detector evaluated in this study is Isolation Forest (IF) [21], a tree-based algorithm that isolates abnormal points rather than modeling normal behavior density. This method was chosen due to its high efficiency and its established effectiveness in network anomaly and intrusion detection [31]–[34]. Each binary tree recursively selects a split variable and a split value at random; anomalies are therefore isolated close to the root, producing shorter path lengths and thus larger anomaly scores. We rely on the high-performance `isotree` library [35], which incorporates recent improvements for faster tree construction and clustered outlier detection.

Seven HPs are tuned because prior work shows they control most of IF’s bias–variance trade-off [36]–[38]. Table III summarizes the search range used for each of them, which defines the configuration search space for θ .

To obtain the optimal configuration, we apply Bayesian optimization (BO) [39] within the configuration search space. BO proceeds as an iterative loop. In our setting, one BO iteration corresponds to: (i) selecting a candidate configuration θ , (ii) training IF with θ on the hyper-parameter tuning set, and (iii) scoring θ by ROC–AUC on a separate validation split. To reduce dependence on any single split, we repeat this train/validate scoring using multiple train/validation splits and use the average ROC–AUC as p_{robust} for BO; the exact splitting and scoring protocol is detailed in Section IV-C. Starting from five quasi-random seed points, BO fits a smooth statistical model to the observed pairs $(\theta, p_{\text{robust}})$ and updates this model after each new iteration. The next candidate is then chosen by maximizing expected improvement over the best score observed so far, balancing exploration and exploitation. We limit the procedure to 200 BO iterations, and the best-observed setting is reported as the approximate optimum θ^* .

C. Evaluation protocol

Figure 3 summarizes our evaluation for a single trace (e.g., one day). The goal is (i) to select hyper-parameters without using the final test fold, and (ii) to estimate ROC–AUC on unseen data. We achieve this using nested 5×4 cross-

validation, which separates hyper-parameter selection from the final performance assessment and avoids selection-induced bias [40]–[42].

Outer loop (5 folds). For a given trace, we partition the benign flows into five outer folds. For each outer iteration $i \in \{1, \dots, 5\}$, we reserve one benign fold as the outer test set and use the remaining four benign folds as the pool for hyper-parameter selection. In parallel, we split the attack flows into five folds. For the hyper-parameter selection phase, we draw one attack fold at random and keep the other four attack folds reserved for the outer evaluation.

HP selection inside the outer loop. Within the hyper-parameter selection pool, BO evaluates candidate configurations θ . To score one candidate θ , we repeatedly train and validate IF within the selection pool: the model is trained on three parts of the benign selection pool and validated on the remaining part, combined with the selected attack fold. We compute ROC–AUC for each validation run and use their average as the objective value p_{robust} for this candidate θ . For each outer iteration, BO is run for 200 iterations and returns the configuration with the highest mean validation ROC–AUC; this configuration is denoted θ_i in Fig. 3.

Outer evaluation After selecting θ_i , we retrain IF on all benign flows in the four outer training folds and evaluate on: (i) the reserved outer benign test fold and (ii) the four attack folds that were not used during hyper-parameter selection. This yields one ROC curve and its corresponding ROC–AUC, denoted ROC_i .

Aggregating across outer folds. Repeating the outer loop for $i = 1, \dots, 5$ produces five values $(ROC_1, \dots, ROC_5)$ and five selected configurations $(\theta_1, \dots, \theta_5)$. For a standard, deployable estimate of performance, we report the mean of the five ROC–AUC values.

Trace-specific oracle upper bound. To report a non-deployable but best-case reference, we adopt test-domain validation (oracle) selection [43]. Among the five selected configurations $\theta_1, \dots, \theta_5$, we choose the one that achieved the highest ROC–AUC on its corresponding outer test fold (the *argmax* in Fig. 3). The corresponding ROC–AUC is reported as the oracle upper bound p_{robust}^* , and the selected configuration is stored in a database of candidate configurations. This follows standard practice in domain generalization benchmarks to quantify the gap between practical methods and an oracle-level upper bound [43]–[45].

The configuration database also contains the default configuration provided by the ML software. When evaluating a configuration taken from the database on a given trace, we keep the same outer evaluation procedure (training on four benign folds and testing on the reserved benign fold plus attacks), but the configuration is fixed and no hyper-parameter search is performed. This produces a mean ROC–AUC that serves as the robustness score of that configuration on the target trace. The next sections specify which trace–configuration pairings are used in each experiment.

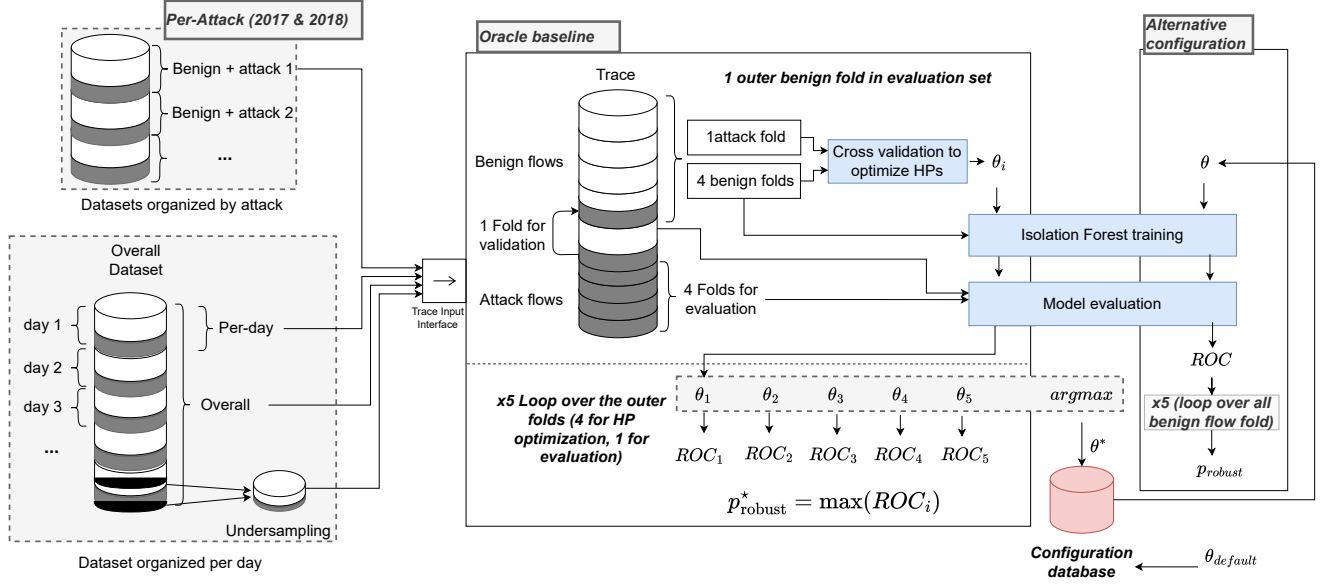


Fig. 3: Evaluation protocol based on nested cross-validation. The top part represents how the configurations (vectors θ) are created or applied and the bottom part represents how datasets are split into different traces.

V. IMPACT OF RECONFIGURATION ON AD IDS PERFORMANCE

This section quantifies the robustness lost when an IF AD IDS is not configured with the well-tuned optimal configuration.

We begin with a *daily analysis* (Section V-A), organized in two parts: The first compares three low-budget choices (*Default*, *Single Overall*, and *Under-sampling* configurations) against the optimal obtained by full Bayesian optimization on the same day's benign flows. The second then examines how well a daily optimized configuration transfers to another day. After this day-centric study, we turn to a *cross-year, per-attack analysis* (Section V-B), where a configuration that is optimal for one attack family in 2017 is applied unchanged to the matching family in 2018.

A. Daily Analysis

1) *Default, Single Overall, and Under-sampling based Configurations:* The first alternative we tested is the *default* configuration, $\theta_{default}$, provided by the libraries implementing anomaly-detection algorithms. This approach represents the most basic option available, requiring no additional technical intervention. In our scenario, we rely on the default configuration offered by the *isotree*¹ library.

A second alternative is a single overall configuration serving as a *one-size-fits-all* configuration. Applying BO to a set of fairly heterogeneous network contexts (a global dataset)

aims to find a configuration suitable for any network-security dataset. The global dataset $\mathbf{D}$ is created as

$$\mathbf{D} = \bigcup_{i=0}^{13} D_i \quad (5)$$

$\mathbf{D}$ includes all 14 days of network traffic from the two datasets. The resulting optimal single overall configuration is $\theta_{overall}^* = \text{BO}(\mathbf{D})$, which can be computed offline. This follows the process described in Fig. 3, in which the trace used is the overall dataset; BO is successively applied to subsets within the nested cross-validation.

Using the same scheme, the final option of our daily analysis, *under-sampling*, involves applying Bayesian optimization on a daily basis. To mitigate the time-consuming nature of this approach, the data are first under-sampled: each day i is sampled at 10% while preserving the class distribution. Under-sampling is applied separately to each traffic label (type). This leads to the following formal definition:

$$\theta_{sample_i}^* = \text{BO}(\text{sample}(D_i, 0.1)), \forall i \in [0, 13] \quad (6)$$

In Fig. 4, each day i is evaluated by applying these derived configuration:

- The default configuration (orange): $p_{robust}(D_i, \theta_{default})$ is calculated.
- The single overall configuration (green): $\theta_{overall}^*$ is distributed daily, and $p_{robust}(D_i, \theta_{overall}^*)$ is calculated.
- The under-sampling configuration (blue): $p_{robust}(D_i, \theta_{sample_i}^*)$ is computed by applying $\theta_{sample_i}^*$ on the corresponding day i .

These curves are benchmarked against the oracle baseline, $p_{robust}(D_i, \theta_i^*)$, which represents the ideal scenario with a daily optimal configuration using BO, depicted in red in Fig. 4.

¹<https://github.com/david-cortes/isotree>

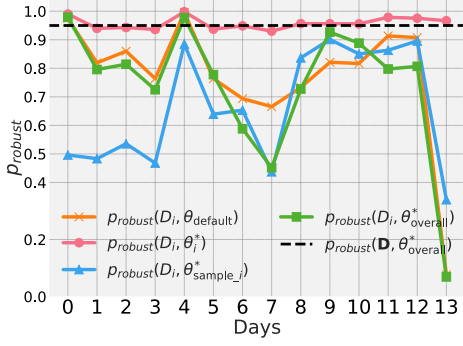


Fig. 4: Default, Overall and Under-sampling

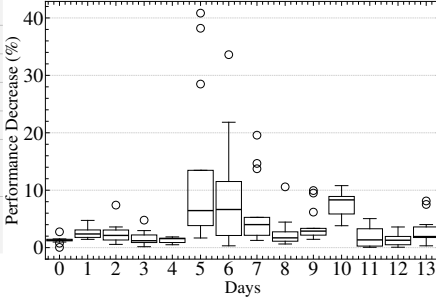


Fig. 5: Daily Transfer of Optimal Configs

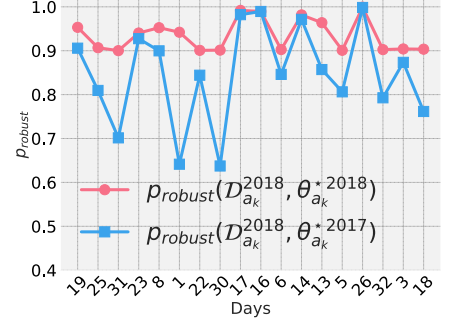


Fig. 6: Transferability of optimal configurations across attacks.

The configuration trained on the entire dataset $\theta_{\text{overall}}^*$ achieves a robustness score of $p_{\text{robust}} = 0.95$, shown as the dashed reference line in the same figure. At first glance, this result seems excellent; however, this is due to a few days with many samples where the overall configuration is well suited. This is highlighted on days 0 and 4 where daily performance using $\theta_{\text{overall}}^*$ (in green) is even higher than 0.95, whereas most of the other days are close to, or significantly below, 0.8. Actually, the average value across all days is just 0.77, with a notable standard deviation of 0.22, underlining the performance disparity of the overall configuration across all days.

In contrast, a daily optimal configuration (in red) allows the AD IDS to be reconfigured so that its performance remains optimal. p_{robust} is always higher than 0.93 for $p_{\text{robust}}(D_i, \theta_i^*)$. Hence, while a configuration like $\theta_{\text{overall}}^*$ may perform well from an aggregated perspective, it does not necessarily ensure effectiveness in every specific context. In our case, a daily-specific configuration outperforms it. Such a result is expected because the configuration was tuned for the specific days to which it is applied, and therefore faces lower data variation originating from the same topology and recurring attacks. This daily-optimized configuration serves as a baseline (ideal case) for subsequent experiments.

By comparison, the default configuration (in orange) only matches the optimal value on days 0 and 4, exhibiting significantly lower values on other days, especially day 13 with the ROC-AUC nearing 0.

Even the under-sampling technique does not provide satisfactory results as highlighted by the curve $p_{\text{robust}}(D_i, \theta_{\text{sample}_i}^*)$ in Fig. 4 (in blue), especially on days 0 to 3, 7, and 13, where p_{robust} is notably below 0.6. Because fewer data are used, the configuration might be less adapted to the heterogeneity of the day's dataset. While under-sampling could have been a simple yet time-efficient alternative to full BO, it is not reliable, as shown in this experiment.

On Day 13, detailed investigation showed that while alternative configurations performed well in certain folds (p_{robust} near 1 for some days), they underperformed in others (p_{robust} near 0). This variability implies Day 13's benign traffic is

quite heterogeneous, making configuration more difficult and contributing to the day's poor results.

2) *Configuration Transferability*: This section considers more intelligent techniques to transfer configurations across days but still simple from a computational perspective. Indeed, one could argue that the configuration learned from one day can be suitable for another one. This section assess whether such an option is viable by testing all possible alternative configurations.

For instance, using daily optimized configurations from D0-D12, we can compute $p_{\text{robust}}(D_{13}, \theta_i^*)$ for each $\theta_0^*, \dots, \theta_{12}^*$ to assess their direct transferability to the 13th dataset. We can then rotate the day of reference by changing the test and compared against the baseline (i.e. the daily optimal configurations). For a given day, we will thus calculate how much the robustness differs from this baseline when applying the configuration of another day.

This is formally defined as:

$$R_i = \{PPD(i, j) \mid \forall j \in [0, 13], i \neq j\}, \forall i \in [0, 13] \quad (7)$$

where R_i denotes the distribution of values obtained from $PPD(i, j)$ and $PPD(i, j)$ is the Performance Decrease Percentage for day i , determined by comparing its optimal AD IDS performance against the performance achieved with day j 's optimal configuration:

$$PPD(i, j) = \frac{p_{\text{robust}}(D_i, \theta_i^*) - p_{\text{robust}}(D_i, \theta_j^*)}{|p_{\text{robust}}(D_i, \theta_i^*)|} \times 100 \quad (8)$$

R_i is shown as box plot in Fig. 5. Days 0-4, 8, 9, 11-13 exhibit median PPD values under 5%, suggesting a viable configuration transfer, despite some outliers nearly reaching or exceeding 10% degradation. In contrast, days 5-7, and 10 exhibit higher scattering, especially days 5 and 6 with values hitting 35-40%. Their wider box plots indicate large performance degradation. In those cases, using thus a configuration of another day should be avoided. Although Day 7 has a compact box, it includes outliers of up to 20%, unlike Day 10, which results in a higher median. Transferability remains inefficient for this day. In conclusion, transferability is practical only on certain days, while it is either impossible or

requires predicting the exact day configuration to transfer for the others. This highlights the challenges in sustaining attack detection performance through inter-day optimal configuration transfers. This invalidates meantime the possibility to have only a few good candidate configurations which allows to maintain IDS performance.

B. Per-attack Analysis

Despite previous findings on inconsistent transferability across days, we now investigate cases where the types of attacks to be detected can be anticipated. We acknowledge that this is an ideal scenario; one might expect that a configuration could be transferred easily across days with similar attacks, because only the network topology and the number of hosts differ between 2017 and 2018. The goal of this section is to verify this hypothesis.

The list $\mathcal{A}$ consists of 18 attacks common to IDS2017 and IDS2018. Each attack a_k in $\mathcal{A}$ ($\forall k \in [0, 17]$) is individually represented in $\mathcal{D}_{a_k}$ for both the 2017 and 2018 versions and is combined with identical benign traffic, randomly sourced from IDS2017. In this section, we present only the assessment of transferability of the optimal configurations from IDS2017 to IDS2018, but similar findings with the reverse scenario have been observed.

With BO, for each $\mathcal{D}_{a_k}^j$ ($j \in \{2017, 2018\}$), we determine the respective $\theta_{a_k}^{*2017}$ and $\theta_{a_k}^{*2018}$ for all $k \in [0, 17]$, followed by computing:

- $\text{Probst}(\mathcal{D}_{a_k}^{2018}, \theta_{a_k}^{*2018})$: the robustness when applying an *optimal* configuration, i.e. specific to the attack and the year (baseline),
- $\text{Probst}(\mathcal{D}_{a_k}^{2018}, \theta_{a_k}^{*2017})$: the robustness when applying an *attack-based* configuration, i.e. specific to the attack but on the previous year.

In Fig. 6, attacks are indexed on the x-axis according to the IDs from Fig. 1. As highlighted, the robustness varies widely among attacks when the configuration is transferred from the previous year (blue). For example, attacks 23, 17, 16, 14, and 26 achieve close values to the baseline (red), minor discrepancies are seen in attacks 19, 8, 22, 6, and 3, but the variance is more significant in attacks 31, 1, and 30.

Even in ideal scenarios where the attack type is known in advance and only the network environment changes, reconfiguring the ML algorithm may still be necessary.

As a reminder, the number of flows is largely higher in IDS2018 than in IDS2017. A higher volume means many additional benign flows, and this affects the Isolation Forest in two ways. First, this reduces the effective subsampling rate, so each tree sees a smaller fraction of the available traffic. Second, deeper trees can be grown because more points are available to split, which increases computational cost and can shift the detector's sensitivity. Both effects imply that a configuration tuned on a light-traffic day may be poorly matched to a heavy-traffic day even when the mix of protocols remains similar. This is confirmed by our experiment.

C. Outcomes

In summary, configuration is critical for AD IDS, where optimal configurations can lead to almost perfect robustness scores measured as averaged ROC-AUC scores. In our experiments, this corresponds to the daily reconfiguration using BO. However, as previously mentioned, this relies on prior knowledge of attack samples, a requirement incompatible with an anomaly-based approach.

We therefore evaluated trivial alternatives by either computing an overall configuration or transferring previous configurations. Neither proves satisfactory and both can significantly degrade performance. The IDS is actually highly sensitive to its configuration and there is no unique alternative scenarios that is always the more suited one.

For example, on day 13, day-based transferability is effective, unlike the overall and undersampling methods. Conversely, on day 5, day-based transferability struggles while the overall method is more effective. Assessing an AD IDS's ability without BO is therefore a relevant and significant challenge.

VI. CONCLUSION

This paper quantifies the performance degradation that an anomaly-detection IDS incurs when its HPs are left unchanged while network traffic patterns or topology change. On the day-partitioned IDS2017 and IDS2018 datasets, a single one-size-fits-all configuration achieves an aggregate ROC-AUC of 0.95 yet delivers only 0.77 ± 0.22 on a day-by-day basis, falling below 0.80 on ten of fourteen days. In contrast, a daily oracle tuned with Bayesian optimization maintains ROC-AUC > 0.93 throughout, while software default values and a 10% under-sampling shortcut collapse on heterogeneous days, dropping to nearly zero on Day 13. Cross-year experiments show that attack-specific configurations tuned on 2017 traffic can lose up to 15 ROC-AUC points in 2018. While this is an unrealistic scenario where the attacks the AD-IDS will face are known in advance, the degradation is still too high to be acceptable. This demonstrates that the optimal HP configuration is highly sensitive to, and impacted by, the unique context in which the AD IDS is deployed.

Continuous, fully automated re-configuration is essential; neglecting it impacts roughly one-third of an AD IDS's discrimination power. Future work will address this paper's problem. We will research adaptive scheduling policies that use a lightweight drift detector to re-configure only when traffic changes significantly.

ACKNOWLEDGMENTS

The authors acknowledge partial support from the French National Research Agency (ANR) through the France 2030 initiative (project Superviz, ANR-22-PECY-0008), and additional support from the Luxembourg National Research Fund (FNR) under grant C23/IS/18088425/COCTEL. The opinions expressed are those of the authors and do not necessarily reflect the views of the French government.

REFERENCES

- [1] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "An efficient approach for assessing hyperparameter importance," in *Proceedings of the 31st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 32. Beijing, China: PMLR, 2014, pp. 754–762. [Online]. Available: <https://proceedings.mlr.press/v32/hutter14.html>
- [2] J. N. van Rijn and F. Hutter, "Hyperparameter importance across datasets," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. London, United Kingdom: ACM, 2018, pp. 1007–1015.
- [3] M. Sugiyama, M. Krauledat, and K.-R. Müller, "Covariate shift adaptation by importance weighted cross validation," *Journal of Machine Learning Research*, vol. 8, pp. 985–1005, 2007.
- [4] C. Cortes, Y. Mansour, and M. Mohri, "Learning bounds for importance weighting," in *Advances in Neural Information Processing Systems* 23, 2010, pp. 442–450.
- [5] J. Kirschner, I. Bogunović, S. Jegelka, and A. Krause, "Distributionally robust bayesian optimization," in *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 108, 2020, pp. 2174–2184. [Online]. Available: <https://proceedings.mlr.press/v108/kirschner20a.html>
- [6] L. Yang, J. Lyu, W. Lyu, and Z. Chen, "Efficient robust bayesian optimization for arbitrary uncertain inputs," *arXiv preprint arXiv:2310.20145*, 2023.
- [7] D. Kim, M. Zecchin, S. Park, J. Kang, and O. Simeone, "Robust bayesian optimization via localized online conformal prediction," *arXiv preprint arXiv:2411.17387*, 2024.
- [8] C. Choi, Y. Lee, A. S. Chen, A. Zhou, A. Raghunathan, and C. Finn, "AutoFT: Robust fine-tuning by optimizing hyperparameters on OOD data," *arXiv preprint arXiv:2401.10220*, 2024, *neurIPS Distribution Shifts Workshop*, 2023.
- [9] B. Veloso, J. Gama, B. Malheiro, and J. Vinagre, "Hyperparameter self-tuning for data streams," *Information Fusion*, vol. 76, pp. 75–86, 2021.
- [10] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*, K. R. B. Butler and K. Thomas, Eds. Boston, MA, USA: USENIX Association, 2022.
- [11] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *28th USENIX Security Symposium (USENIX Security 19)*, N. Heninger and P. Traynor, Eds. Santa Clara, CA, USA: USENIX Association, 2019, pp. 729–746. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury>
- [12] M. Masum, H. Shahriar, H. Haddad, M. J. H. Faruk, M. Valero, M. A. Khan, M. A. Rahman, M. I. Adnan, A. Cuzzocrea, and F. Wu, "Bayesian hyperparameter optimization for deep neural network-based network intrusion detection," in *Proceedings of the 2021 IEEE International Conference on Big Data*. Orlando, FL, USA: IEEE, 2021, pp. 5413–5420.
- [13] O. Anser, J. François, and I. Chrismont, "Automated machine learning configuration to learn intrusion detectors on attack-free datasets," in *2024 IEEE 49th Conference on Local Computer Networks (LCN)*, 2024.
- [14] Q. Lin, Z. Liu, Y. Yang, K.-C. Wong, Y. Lu, and J. Li, "Multi-objective evolutionary neural architecture search for network intrusion detection," *Swarm and Evolutionary Computation*, vol. 91, p. 101702, 2024.
- [15] J. Shao, G. Zeng, K. Lu, G. Geng, and J. Weng, "Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search," *Computers & Security*, vol. 143, p. 103910, 2024.
- [16] A. Lakhina, M. Crovella, and C. Diot, "Mining anomalies using traffic feature distributions," in *Proceedings of SIGCOMM '05*, 2005, pp. 217–228.
- [17] D. Wagner and P. Soto, "Mimicry attacks on host-based intrusion detection systems," in *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, 2002, pp. 255–264.
- [18] P. Fogla and W. Lee, "Evading network anomaly detection systems: Formal reasoning and practical techniques," in *Proceedings of the 13th ACM Conference on Computer and Communications Security (CCS)*, 2006, pp. 59–68.
- [19] J. Cao, W. S. Cleveland, D. Lin, and D. X. Sun, "On the nonstationarity of internet traffic," in *Proceedings of the 2001 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS '01)*. New York, USA: Association for Computing Machinery, 2001, pp. 102–112.
- [20] A. Feldmann, A. C. Gilbert, W. Willinger, and T. G. Kurtz, "The changing nature of network traffic: Scaling phenomena," *ACM SIGCOMM Computer Communication Review*, vol. 28, no. 2, pp. 5–29, Apr. 1998.
- [21] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [22] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *2010 IEEE Symposium on Security and Privacy (SP '10)*, Oakland, CA, USA, 2010, pp. 305–316.
- [23] O. Anser, J. François, and I. Chrismont, "Auto-tuning of hyperparameters for detecting network intrusions via meta-learning," in *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, 2023, pp. 1–6.
- [24] I. e. a. Sharafaldin, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *4th International Conference on Information Systems Security and Privacy*, 2018.
- [25] I. Gulrajani and D. Lopez-Paz, "In search of lost domain generalization," *arXiv preprint arXiv:2007.01434*, 2020.
- [26] P. W. Koh, S. Sagawa, H. Marklund, S. M. Xie, M. Zhang, A. Balsubramani, W. Hu, M. Yasunaga, R. L. Phillips, I. Gao, T. Lee, E. David, I. Stavness, W. Guo, B. Earnshaw, I. Haque, S. Beery, J. Leskovec, A. Kundaje, E. Pierson, S. Levine, C. Finn, and P. Liang, "Wilds: A benchmark of in-the-wild distribution shifts," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 5637–5664. [Online]. Available: <https://proceedings.mlr.press/v139/koh21a.html>
- [27] S. Zhang, Y. Wu, Z. Zheng, Q. Wu, and C. Wang, "Hypertime: Hyperparameter optimization for combating temporal distribution shifts," in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024.
- [28] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in nids datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proceedings of the 2022 IEEE Conference on Communications and Network Security (CNS)*. Austin, TX, USA: IEEE, Oct. 2022, pp. 254–262.
- [29] P. Jairu and A. B. Mailewa, "Network anomaly uncovering on cids-2017 dataset: A supervised artificial intelligence approach," in *Proceedings of the IEEE International Conference on Electro/Information Technology (EIT)*. IEEE, 2022, pp. 347–352, feature-selection study that isolates 14 most discriminative NetFlow-style attributes.
- [30] E. M. Zeleke, H. M. Melaku, and F. G. Mengistu, "Efficient intrusion detection system for sdn orchestrated internet of things," *Journal of Computer Networks and Communications*, vol. 2021, pp. 1–15, 2021, recursive-feature-elimination shows best accuracy with the same 14 flow features.
- [31] R. Li, Q. Li, Y. Zhang, D. Zhao, X. Xiao, and Y. Jiang, "Genos: General in-network unsupervised intrusion detection by rule extraction," in *IEEE INFOCOM 2024*, 2024, accepted by INFOCOM 2024; preprint arXiv:2403.19248. [Online]. Available: <https://arxiv.org/abs/2403.19248>
- [32] D. Barradas, N. Santos, and L. Rodrigues, "Effective detection of multimedia protocol tunneling using machine learning," in *27th USENIX Security Symposium*, 2018. [Online]. Available: <https://www.usenix.org/system/files/conference/usenixsecurity18/sec18-barradas.pdf>
- [33] S. Mittal, H. V., P. Heetkumar, and P. Tammana, "iguard: Efficient isolation forest design for malicious traffic detection in programmable switches," in *Proceedings of the 20th International Conference on emerging Networking Experiments and Technologies (CoNEXT '24)*. Los Angeles, CA, USA: ACM, 2024. [Online]. Available: <https://conferences.sigcomm.org/co-next/2024/files/papers/p55.pdf>
- [34] Y. Dong, Q. Li, K. Wu, R. Li, D. Zhao, G. Tyson, J. Peng, Y. Jiang, S. Xia, and M. Xu, "Horuseye: A realtime iot malicious traffic detection framework using programmable switches," in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA, USA, 2023, pp. 571–588. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity23-dong-yutao.pdf>
- [35] D. Cortes, "isotree: Isolation-based outlier detection," <https://isotree.readthedocs.io/en/latest/>, 2023.
- [36] S. Hariir, M. Carrasco Kind, and R. J. Brunner, "Extended isolation forest," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1479–1489, 2019.

- [37] D. Cortes, "Revisiting randomized choices in isolation forests," *arXiv preprint arXiv:2110.13402*, 2021.
- [38] Y. Chabchoub, M. U. Togbé, A. Boly, and R. Chiky, "An in-depth study and improvement of isolation forest," *IEEE Access*, vol. 10, pp. 10 219–10 237, 2022.
- [39] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, 2012, pp. 2960–2968.
- [40] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, no. 1, p. 91, 2006.
- [41] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. [Online]. Available: <https://www.jmlr.org/papers/volume11/cawley10a/cawley10a.pdf>
- [42] S. Arlot and A. Celisse, "A survey of cross-validation procedures for model selection," *Statistics Surveys*, vol. 4, pp. 40–79, 2010.
- [43] I. Gulrajani and D. Lopez-Paz, "In search of lost domain generalization," in *International Conference on Learning Representations*, 2021, <https://openreview.net/forum?id=IQdXeXDoWtI>.
- [44] D. Kim, Y. Yoo, S. Park, J. Kim, and J. Lee, "Selfreg: Self-supervised contrastive regularization for domain generalization," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2021, pp. 9619–9628.
- [45] "Supplemental material to selfreg: Self-supervised contrastive regularization for domain generalization," https://openaccess.thecvf.com/content/ICCV2021/supplemental/Kim_SelfReg_Self-Supervised_Contrastive_ICCV_2021_supplemental.pdf, 2021, supplemental material.